Project 2: Nano Games

ECE 2564 – Embedded Systems

Virginia Polytechnic Institute of Technology

Benjamin Adjepong

April 11th, 2023

GitHub Username: benjivt

GitHub Repository: https://github.com/ECE2564-S23/proj2-hw9-hw10-benjivt

Report Summary

The objective of this report is to present the details of the Nano Games project. This project includes three games that cycle through each other if a player has enough lives. These games were developed by implementing finite state machines and pointers to application struct variables.

Project Description

The Nano Game project consists of three games: Meter Filler, Direction Obeyer and Color Confirmer. Each game has a pregame screen that lasts for 2 seconds before the game starts and lasts for 4 seconds. The Meter Filler game starts with an empty rectangle on the screen that is gradually filled during the game period. The goal of this game is the push the boosterpack button 1 within half a second of the rectangle filling up. If the button is pressed on time a success message is displayed and the sqaures in the rectangle turn green. If the button is pressed too early or not on time, a failure message is displayed and the squares in the game turn red. In the Direction Obeyer game, the player is prompted to hold the joystick in a random direction. If the the player holds the joystick in the indicated direction , a success message is displayed, and a failure message is displayed otherwise. For the last game, Color Confirmer, both a random color square and random statement coreesponding to the square are displayed to the screen. If the statement is true and the player presses the boosterpack button 1 (input for true), a success message is displayed, and likewise when the statement is false and boosterpack button 2 (input for false) is pressed. After each successful game, the score count is incremented by 1000 but after each failed game the live count is decremented. The Nano Games come to an end when all 3 of the player's lives are used up. From there, the game over screen is hown displaying the score of that game session. In this screen the high scores are also updated based on the score of the game session. Below are all the different screens used in this project.

- Title Screen, Pregame Screen, Game Over Screen

Figure 1: Title Screen

Figure 2: Pregame Screen

Figure 1: Game Over

- Menu Screen, Instruction Play Screen, HighScore Screen

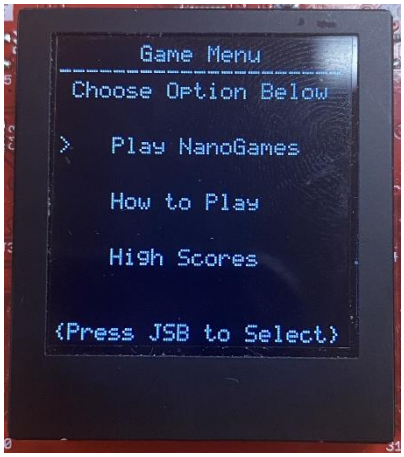


Figure 4: Menu Screen

Figure 5: Instructions Screen

Figure 6: High Score Screen

- Meter Filler

Figure 7: Meter Filler before input

Figure 8: Meter Filler Success Message

Figure 9: Meter Filler Failure Message

- Direction Obeyer

Figure 10: Direction Obeyer Success Message

Figure 11: Direction Obeyer Failure Message

- Color Confirmer

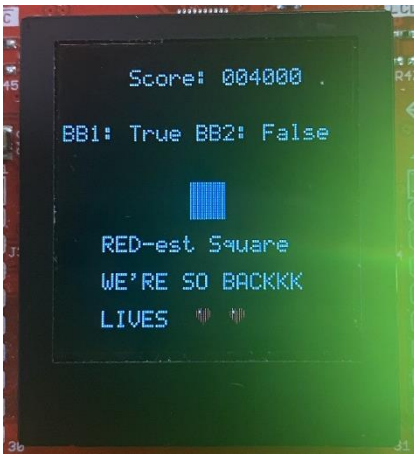
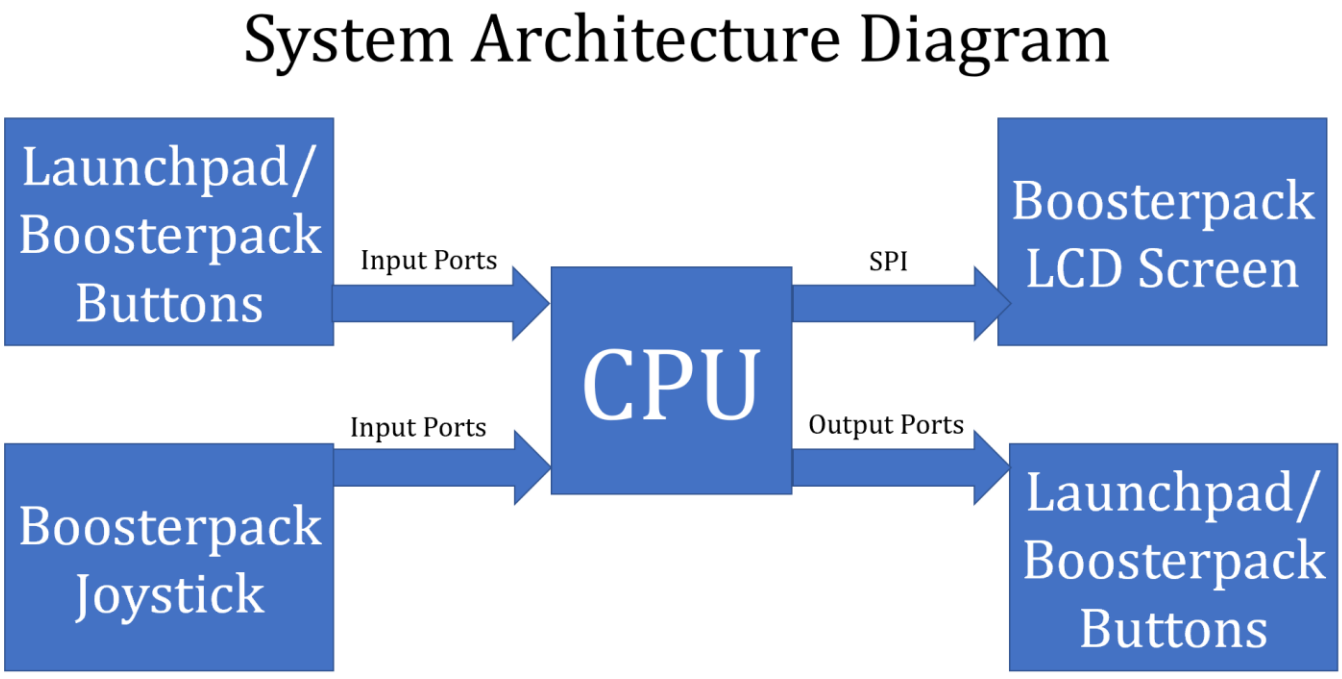


Figure 12: Color Confirmer Success Message

Figure 13: Color Confirmer Failure Message

System Architecture

The system architecture for this project consists of SPI communication protocol and GPIO connections for data transmission/receiving. Ther GPIO port and pins are used to receive inputs from peripherals such as the launchpad buttons and the Boosterpack buttons and joystick. Likewise, the GPIO ports and pins are also used to send outputs to peripherals such as launchpad and Boosterpack LEDs. The SPI comm protocol is serial and full duplex meaning that data can be transmitted and received in two directions simultaneously. The SPI protocol established a connection with the Boosterpack LCD screen and enabled graphics to be displayed on it.



Code Quality

It is expected that the project was developed in high quality code since well-written code can be easily tested, updated, and shared. For this project, the quality of the code is evaluated as shown below.

	Description	Expected Score
Comments	Code is reasonably commented so that any person can understand the code	10/10
No Global Variables	No global variables are used, except for custom images using Image Reformer	20/20
No Numeric Values	Macros and constants are used instead of hard-coded numbers, except for UI/Graphics elements	20/20
No Long Functions	No function exceeds 50 lines of code, within reason. Comment lines and empty lines do not contribute to this total.	30/30
Non-blocking code	Code is completely non-blocking. Holding down LB1 lights up Launchpad LL1 at any time.	20/20

Bonus Points

This project had several opportunities to score additional bonus points. The bonus sections that were implemented were custom images for the title screen, live icons and other screens. The images for these parts were obtained from free PNG websites online. Additionally the color craziness feature was implemented by turning on the Boosterpack LED to a randomly generated color during the Color Confirmer Nano Game to add a layer of confusion and chaos. Lastly, all the random numbers were generated using an ADC random generator instead of the rand() function. This was implemented by sampling the x and y inputs from the joystick and using a simple random generator algorithm using the already generated noise from the joystick ADC.

Bonus	Description	Expected Points
Title Screen	Implemented static custom image for title screen	5/15
Live Icons	Used custom images for live icons	15/15
ADC Random Generator	Instead of using rand functions to generate random numbers, the noise from the joystick input to generate random numbers	25/25
Color Craziness	During Color Confirmer turn on random colored LED to add layer of chaos	15/15
Custom Graphics	Image on color screen (5 points) Different image depending on win or loss (Direction – 10 points)	15/25